Kuantum Süperpozisyon Mekaniği ve "Schrödinger'in Kedisi" Temalı Mobil Oyun Teknik Fizibilite ve Mimari Tasarım Raporu (2026)

1. Giriş ve Pazar Analizi

Mobil oyun endüstrisi, 2026 yılı itibarıyla donanım kapasitelerindeki artış ve oyuncu beklentilerinin evrimi ile birlikte, basit "dokun-geç" mekaniklerinden, zihinsel derinliği olan ve görsel olarak zenginleştirilmiş deneyimlere doğru radikal bir dönüşüm geçirmektedir. Sensor Tower ve Newzoo tarafından yayınlanan 2026 pazar raporları, küresel mobil oyun gelirlerinin uygulama içi satın alımlar (IAP) ve hibrit gelir modelleriyle yıllık %10,6 artışla 167 milyar dolara ulaştığını göstermektedir.[1] Bu büyüme, özellikle "puzzle-platformer" (bulmaca-platform) türünde, oyuncuyu entelektüel olarak zorlayan ve yenilikçi mekanikler sunan oyunlara olan talebi artırmıştır. Geleneksel oyun döngülerinin doygunluğa ulaştığı bir pazarda, kuantum fiziğinin temel prensiplerini oyunlaştıran "Schrödinger'in Kedisi" konsepti, hem narratif derinliği hem de teknik yenilik potansiyeli ile öne çıkan bir "Mavi Okyanus" stratejisi sunmaktadır.

Bu rapor, Avusturyalı fizikçi Erwin Schrödinger'in 1935 yılında ortaya attığı ünlü düşünce deneyini [2] temel alan, oyuncunun süperpozisyon durumundaki bir kediyi (Mittens) kontrol ederek, gözlemci etkisi altında gerçeklikleri manipüle ettiği bir mobil oyunun teknik, mimari ve tasarımsal fizibilitesini en ince detayına kadar incelemektedir. Oyunun temel vaadi, kuantum mekaniğinin soyut ve paradoksal doğasını—süperpozisyon, dolanıklık ve dalga fonksiyonunun çökmesi—dokunmatik ekranlar için optimize edilmiş somut oyun mekaniklerine dönüştürmektir.[3]

Teknik açıdan bu proje, 2026 yılı standartlarında mobil cihazların (iOS/Android) grafik işleme ünitelerinin (GPU) sınırlarını zorlayan "Dual World Rendering" (Çift Dünya Görselleştirmesi) tekniklerini gerektirmektedir. Godot 4.5 ve Unity 6 (Universal Render Pipeline - URP) gibi oyun motorlarının sunduğu yeni nesil Stencil Buffer (Şablon Tamponu) ve Render Feature yetenekleri, bu tür karmaşık görselleştirmelerin mobil cihazlarda yüksek performansla (60 FPS) çalıştırılmasını mümkün kılmaktadır.[4] Ayrıca, oyunun asenkron çok oyunculu yapısını destekleyecek olan "Hayalet Veri" (Ghost Data) mimarisi, Nakama ve PlayFab gibi Backend-as-a-Service (BaaS) çözümleri üzerinden deterministik veri senkronizasyonunu zorunlu kılmaktadır.[7]

Rapor, kavramsal tasarımdan başlayarak, frontend grafik mühendisliği, backend veri mimarisi, kullanıcı deneyimi (UX) ve performans optimizasyonu konularını, bir oyun mimarı ve teknik

yönetmen perspektifiyle, akademik titizlikle ve endüstriyel uygulama derinliğiyle ele alacaktır.

---

2. Teorik Çerçeve: Kuantum Mekaniğinin Oyunlaştırılması

Oyun tasarımının temelini oluşturan fizik kuralları, sadece bir "tema" olmanın ötesinde, oyunun çekirdek mekaniklerini (core loop) belirleyen kurallar bütünüdür. Schrödinger'in kedisi paradoksu, bir kuantum sisteminin (kedi), dışarıdan bir gözlemci tarafından ölçüm yapılana kadar tüm olası durumlarda (hem canlı hem ölü) aynı anda bulunabildiğini öne sürer.[2] Bu teorik altyapı, oyun mekaniğine şu şekilde entegre edilmektedir:

2.1 Süperpozisyon (Superposition) ve Durum Yönetimi

Oyuncu, tek bir karakteri kontrol ederken, bir tetikleyici (örneğin "Split" butonu) aracılığıyla kediyi iki farklı olasılık dalgasına ayırır. Bu durumda oyun evreni, "Alfa Durumu" (Canlı/Mavi) ve "Beta Durumu" (Ölü/Kırmızı) olarak ikiye ayrılır.

- **Mekanik:** Oyuncu, aynı anda iki karakteri kontrol eder. Ancak bu karakterler fiziksel olarak farklı özelliklere sahiptir. Alfa kedi fiziksel nesneleri (kutular, anahtarlar) itebilirken, Beta kedi (Hayalet form) katı maddelerin içinden geçebilir (Kuantum Tünelleme) ancak nesnelerle etkileşime giremez.[3]
- **Oyunlaştırma:** Bu durum, oyuncunun bir engeli aşmak için kediyi ikiye bölmesini, Beta kediyle duvarın arkasına geçip bir düğmeye basmasını (etkileşim kısıtı olsa da, enerji bazlı tetikleyiciler kullanılabilir) ve ardından tekrar birleşmesini gerektiren bulmacalar doğurur.

2.2 Gözlemci Etkisi (The Observer Effect) ve Dalga Fonksiyonunun Çökmesi

Kuantum mekaniğinde bir sistemin ölçülmesi, süperpozisyonu bozar ve sistemi tek bir duruma "çöker" (collapse).[2] Oyun tasarımında bu prensip, düşman veya engel mekaniği olarak kullanılır.

- **Gözlemci (Observer):** Güvenlik kameraları, lazer sensörleri veya robotik düşmanlar "Gözlemci" rolünü üstlenir.
- **Çökme (Collapse):** Eğer bir Gözlemci, süperpozisyon halindeki kedilerden birini (örneğin Alfa'yı) görürse, evren anında o duruma çöker. Diğer olasılık (Beta kedi) "teorik" hale gelir ve varlığı silinir. Eğer oyuncu, Beta kedinin bulunduğu konumda hayatta kalması gereken bir strateji kurduysa (örneğin Alfa kedi bir tuzağın üzerindeyse), gözlem anı oyunun kaybedilmesine neden olur.[3] Bu mekanik, oyuncuyu "gözlemlenmeme" üzerine kurulu bir gizlilik (stealth) stratejisine iter.

2.3 Kuantum Dolanıklığı (Quantum Entanglement)

İki parçacığın birbirine bağlı olması ve birinin durumunun diğerini anında etkilemesi prensibidir.

- **Mekanik:** İleri seviyelerde, iki farklı süperpozisyon halindeki kedi birbirine "dolanık" hale getirilebilir. Bir kedi zıpladığında diğeri eğilir veya bir kedi sağa gittiğinde diğeri sola gider (Ters kinematik kontrol). Bu, koordinasyon ve zamanlama odaklı "tek dokunuşla çift kontrol" bulmacaları için zemin hazırlar.

---

3. Frontend Mühendisliği: Grafik Mimarisi ve Render Teknikleri

Projenin en büyük teknik zorluğu, mobil cihazların kısıtlı donanım kaynaklarında (Tile-Based Deferred Rendering - TBDR mimarisi) iki farklı gerçekliği aynı anda, yüksek kare hızında (60 FPS) ve görsel bütünlükle render etmektir. Bu bölümde Godot 4.5 ve Unity 6 motorlarının yetenekleri ve "Dual World" implementasyonu detaylandırılacaktır.

3.1 Dual World Rendering ve Stencil Buffer Teknolojisi

Oyuncunun aynı sahnede iki farklı durumu (Canlı ve Ölü) görmesi veya bir "portal" penceresinden diğer durumu gözlemlemesi için **Stencil Buffer** (Şablon Tamponu) kullanımı endüstri standardı ve performans açısından en verimli yöntemdir.[4] Stencil Buffer, derinlik tamponu (Depth Buffer) ile birlikte çalışan, her piksel için 8 bitlik (0-255 arası) bir maskeleme değeri saklayan bir veri yapısıdır.

3.1.1 Godot 4.5 ile Stencil Buffer İmplementasyonu

Godot 4.5 sürümü, daha önceki sürümlerde eksik olan kapsamlı Stencil Buffer desteğini RDPipelineDepthStencilState sınıfı ve shader seviyesinde stencil_mode ile getirmiştir.[5] Bu, özellikle Vulkan tabanlı "Mobile" ve "Forward+" render motorlarında devrim niteliğindedir.

Teknik Kurulum:

1. **Render Katmanları (Render Layers):**
   Sahne, "Ortak Dünya", "Alfa Dünyası" ve "Beta Dünyası" olarak katmanlara ayrılır. VisualInstance3D üzerindeki layers özelliği bit maskeleriyle yönetilir (Örn: Layer 1: Ortak, Layer 2: Alfa, Layer 3: Beta).
2. **RDPipelineDepthStencilState Konfigürasyonu:**
   Godot'un RenderingDevice API'si üzerinden stencil operasyonları şu şekilde yapılandırılır:
   - **Maske Objesi (Portal/Pencere):** enable_stencil aktif edilir. front_op_pass (Ön yüz işlemi) STENCIL_OP_REPLACE olarak ayarlanır. front_op_reference değeri 1 verilir. Bu obje çizildiğinde, ekrandaki piksellerin stencil değerini 1 yapar. Renk yazma (Color Write) kapalıdır, sadece stencil tamponuna yazar.[5]
   - **İçerik Objeleri (Beta Dünyası):** Bu objelerin materyallerinde stencil_mode kullanılarak read (okuma) işlemi yapılır. front_op_compare (Karşılaştırma operatörü)

COMPARE_OP_EQUAL olarak ayarlanır ve referans değeri 1 verilir.
    - **Sonuç:** Beta dünyasındaki objeler, sadece stencil değerinin 1 olduğu (yani maske objesinin çizildiği) piksellerde görünür hale gelir.

Godot Shader Kodu Örneği (Spatial Shader):

OpenGL Shading Language

```
shader_type spatial;
render_mode stencil_test_enabled; // Stencil testini aç

// Stencil parametreleri (Godot 4.5 sentaksı)
// Bu materyal sadece Stencil değeri 1 olan yerlerde çizilir.
void fragment() {
  // Görselleştirme kodları
  ALBEDO = vec3(1.0, 0.0, 0.0); // Kırmızı (Beta Durumu)
}
```

Not: Godot 4.5'te render_mode içine stencil_compare_equal ve stencil_reference_1 gibi doğrudan direktifler veya inspector üzerinden RDPipelineDepthStencilState ayarları kullanılabilir.[12]

3.1.2 Unity 6 (URP) ile Render Objects Feature Kullanımı

Unity'nin Universal Render Pipeline (URP) mimarisi, Renderer Features özelliği ile render döngüsüne müdahale etmeyi sağlar. Bu yöntem, shader kodlamasına girmeden, editör arayüzü üzerinden karmaşık stencil işlemlerini yönetmeyi mümkün kılar.[13]

Adım Adım Mimari Kurulum:

1. **Forward Renderer Data Ayarları:**
   URP Asset dosyasında, "Render Objects" özelliğini ekleyin. Buna "QuantumMask_Write" adını verin.
    - **Event:** AfterRenderingOpaques (Opak objelerden sonra).
    - **Filters:** Queue: Transparent, Layer Mask: MaskLayer.
    - **Overrides -> Stencil:**
      - Value: 1
      - Compare Function: Always (Her zaman yaz).
      - Pass Op: Replace (Mevcut değeri 1 ile değiştir).
      - Z Test: Less Equal (Derinlik testini uygula).

2. **İkinci Render Feature (Okuma):**
   Yeni bir "Render Objects" özelliği ekleyin: "QuantumWorld_Read".
   - **Filters:** Layer Mask: BetaWorldLayer.
   - **Overrides -> Stencil:**
     - Value: 1
     - Compare Function: Equal (Sadece değer 1 ise çiz).
     - Pass Op: Keep (Değeri koru).

Mobil Optimizasyon (Unity URP): Mobil cihazlarda performans kaybını önlemek için Depth Texture Mode ayarı "After Transparents" olarak seçilmelidir. Bu, URP'nin opak ve transparan geçişleri arasında gereksiz "Render Target" değişimlerini (context switching) engeller.[14] Ayrıca, Render Scale 0.8 veya 0.75 değerine çekilerek, piksel dolum oranı (fill rate) düşürülmeli ve mobil GPU'nun ısınması engellenmelidir.[15]

3.2 Görsel Efektler: Kuantum Belirsizliği ve Glitch Shaderları

Oyunun görsel dilinde, kedinin süperpozisyon durumu "stabil olmayan" bir gerçeklik olarak tasvir edilmelidir. Bu, statik bir görüntü yerine, sürekli titreyen, bozulan ve yeniden birleşen bir "Glitch" estetiği gerektirir.

3.2.1 Vertex Displacement ve Gürültü (Noise) Fonksiyonları

Kuantum dalgalanmasını simüle etmek için karakterin mesh yapısındaki vertex'ler (köşe noktaları) dinamik olarak hareket ettirilmelidir. Shader Graph (Unity) veya VisualShader (Godot) kullanılarak:

- **Teknik:** Vertex Shader aşamasında, vertex pozisyonlarına Simplex Noise veya Voronoi Noise eklenir. Gürültü fonksiyonu TIME (zaman) değişkeni ile kaydırılarak (scrolling) hareketlilik sağlanır.
- **Mantık:**

$$P_{yeni} = P_{eski} + (Nornal \times Noise(P_{eski} + Zaman) \times Şiddet)$$

  Bu formül, karakterin yüzeyinde sanki sıvıymış veya enerji dalgasıymış gibi kabarmalar yaratır.[16]

3.2.2 Ekran Uzayı (Screen Space) Efektleri ve AGSL

Tüm ekranı etkileyen "Çökme" veya "Dolanıklık" anları için post-processing efektleri kullanılır. Android 13 ve üzeri cihazlarda, **Android Graphics Shading Language (AGSL)** kullanılarak, UI elementleri üzerinde yüksek performanslı programlanabilir shaderlar çalıştırılabilir.

- **Kromatik Sapma (Chromatic Aberration):** Gözlemci kamerası kediyi gördüğünde, ekranın RGB kanalları birbirinden ayrılmalıdır. Kırmızı kanal sağa, Mavi kanal sola kaydırılarak "gerçekliğin yırtılması" hissi verilir.
- **Scanline ve Pixelation:** Retro-fütüristik bir hava katmak için ekrana tarama çizgileri ve

anlık pikselleşme (resolution downscaling) uygulanır. Bu efektler, SceneColor dokusu alınıp, UV koordinatları manipüle edilerek (örneğin floor(UV * Resolution) / Resolution) elde edilir.[18]

---

4. Backend Mimarisi: Asenkron Multiplayer ve Veri Yönetimi

Oyunun sosyal katmanını, oyuncuların arkadaşlarının bölüm çözümlerini "Hayalet" (Ghost) olarak izleyebilmesi oluşturur. Bu, gerçek zamanlı (senkron) bir multiplayer değil, asenkron bir veri tekrarı sistemidir.

4.1 Hayalet Veri (Ghost Data) Kayıt Stratejileri

Mobil ağlarda veri kullanımını minimize etmek ve sunucu maliyetlerini düşürmek için veri optimizasyonu kritiktir. İki ana yöntem mevcuttur:

4.1.1 Yöntem A: Deterministik Girdi Kaydı (Input Recording)

Sadece oyuncunun tuş basışlarını (Input) ve zamanlamalarını kaydeder.

- **Veri Yapısı:** [{t:0.1, key:"jump"}, {t:0.5, key:"right"}]
- **Risk:** Mobil cihazların işlemcileri (CPU) arasındaki kayan nokta (floating point) hesaplama farkları, simülasyonun determinizmini bozabilir. Bir Samsung cihazda 1.00001 birim zıplayan kedi, iPhone'da 1.00002 zıplarsa, 30 saniye sonra hayalet kedi duvara çarpabilir.[20]
- **Çözüm:** Platformer oyunlarında tam determinizm zordur. Bu yöntem önerilmez.

4.1.2 Yöntem B: Durum Örnekleme ve Sıkıştırma (State Sampling & Compression)

Karakterin pozisyon (Vector3) ve rotasyon (Quaternion) verileri belirli aralıklarla (Tick Rate: 10Hz veya 20Hz) kaydedilir.

- **Delta Sıkıştırma (Delta Compression):** Her karede tam pozisyonu saklamak yerine, sadece bir önceki kareye göre değişim farkı saklanır. Değişim yoksa veri yazılmaz.
- **İnterpolasyon:** İstemci tarafında (Client), 10Hz gelen veri, Lerp (Linear Interpolation) veya Slerp (Spherical Linear Interpolation) fonksiyonları ile 60 FPS akıcı harekete dönüştürülür.[21]
- **Önerilen Format (Protobuf/Binary):** JSON yerine Protocol Buffers veya özel Binary format kullanılarak veri boyutu %70 oranında azaltılabilir.

4.2 BaaS (Backend-as-a-Service) Seçimi: Nakama vs. PlayFab

2026 yılı ekosisteminde, özellikle Godot ve Unity ile uyumluluk açısından iki ana sağlayıcı öne çıkmaktadır: Heroic Labs **Nakama** ve Microsoft Azure **PlayFab**.[7]

Özellik	Nakama (Heroic Labs)	Microsoft Azure PlayFab	Google Firebase
Mimari	Açık Kaynak, Go/TypeScript Runtime, Self-Hosted veya Managed Cloud.	Kapalı Kaynak, Serverless CloudScript (C#/JS).	Realtime Database / Firestore.
Hayalet Veri	**Storage Engine** ile binary obje saklama. Yüksek performanslı okuma/yazma.[8]	Entity Objects veya Title Data. Dosya boyutu limitleri olabilir.	Büyük veri setleri için maliyetli (Okuma/Yazma başına ücret).
Otorite	Sunucu tarafı otoriter (Authoritative) mantık kurmak için yerleşik destek.[8]	Azure Functions ile mantık kurulabilir, ancak "Cold Start" gecikmesi olabilir.	Sınırlı doğrulama (Validation) yeteneği.
Uyumluluk	Godot ve Unity için resmi SDK desteği mükemmel.[24]	Unity SDK güçlü, Godot desteği topluluk tabanlı olabilir.	Mobil odaklı, oyun motoru entegrasyonu daha genel.

Öneri ve Mimari Karar: Proje için **Nakama** tercih edilmelidir. Açık kaynak olması, Docker ile istenilen sunucuda (AWS, Google Cloud, DigitalOcean) barındırılabilmesi ve özellikle **Storage Engine** yapısının "Ghost Data" gibi büyük binary verileri (blob) saklamak için optimize edilmiş olması belirleyici faktördür.[26]

4.3 Otoriter Veri Doğrulama ve Güvenlik

Oyuncuların sahte skor veya imkansız hayalet verileri göndermesini engellemek için "Authoritative" (Otoriter) bir doğrulama sistemi kurulmalıdır.

İş Akışı (Workflow):

1. **Kayıt:** İstemci (Client), bölüm bittiğinde sıkıştırılmış hayalet verisini ve bölüm bitirme süresini Nakama sunucusuna gönderir.
2. **RPC (Remote Procedure Call):** Sunucuda çalışan bir Go veya TypeScript fonksiyonu (RPC) bu veriyi karşılar.[24]

3. **Doğrulama (Validation):** Sunucu, basit bir "Sanity Check" (Mantık Kontrolü) yapar. Örn: Bölümün en hızlı teorik bitiş süresi 30 saniye ise ve gelen veri 5 saniye ise, veri reddedilir.
4. **Depolama (Storage Write):** Geçerli veri, StorageWrite fonksiyonu ile veritabanına yazılır. İzinler (PermissionRead: 1, PermissionWrite: 0) ayarlanarak, verinin sadece sunucu tarafından değiştirilebileceği, ancak herkes tarafından okunabileceği garanti altına alınır.[8]

---

5. Kullanıcı Deneyimi (UX) ve Arayüz Tasarımı (2026 Trendleri)

2026 mobil tasarım trendleri, "Diegetic" (Hikaye içi) ve "Minimalist" arayüzlerin yükselişine işaret etmektedir. Oyuncuyu arayüz elemanlarıyla (butonlar, barlar) boğmak yerine, bilginin oyun dünyasına yedirilmesi hedeflenir.

5.1 Diegetic UI (Oyun İçi Arayüz) Tasarımı

Geleneksel HUD (Heads-Up Display) yerine, oyunun evreniyle bütünleşmiş göstergeler kullanılmalıdır.[27]

- **Süperpozisyon Enerjisi:** Ekranın üstünde bir "Mana Bar" yerine, kedinin üzerindeki "Kuantum Tasması"nın ışık şiddeti veya rengi (Maviden Kırmızıya dönüş) kalan enerjiyi göstermelidir. Enerji azaldığında kedinin kendisi de "glitch"lenmeye ve şeffaflaşmaya başlamalıdır.
- **Gözlemci Uyarısı:** Kamera kediyi gördüğünde, ekranın kenarlarında kırmızı bir vignette (kararma) yerine, doğrudan oyun dünyasındaki ışıklar tepki vermelidir. Kameranın lazeri kedinin üzerine kilitlendiğinde, kedinin gölgesi "titremeye" başlamalı ve "çökme" riski görselleştirilmelidir.[29]

5.2 Kontrol Şeması: "Tek Dokunuş" (One-Tap) Devrimi

Mobil platformlarda karmaşık sanal joystickler (Virtual Joystick) oyuncu deneyimini baltalayabilir. "Split Masters" ve "One More Button" gibi oyunlardan ilhamla, basitleştirilmiş bir kontrol şeması uygulanmalıdır.[31]

- **Hareket:** Otomatik koşu (Auto-runner) veya "Basılı Tut - Koş / Bırak - Dur" mekaniği.
- **Bölünme (Split):** Ekrana tek bir dokunuş (Tap), karakteri süperpozisyona sokar.
- **Karakter Seçimi:** Parmağı yukarı kaydırma (Swipe Up) Alfa ve Beta kediler arasında geçiş yapar.
- **Etkileşim:** Yakındaki nesnelerle etkileşim bağlamsaldır (Contextual). Karakter kutunun yanındaysa otomatik iter.

5.3 Görsel Stil: Neumorphism 2.0 ve Cam Estetiği

2026 arayüz trendleri, "Glassmorphism" (Buzlu Cam) ve "Neumorphism"in evrimleşmiş halidir. Bu stil, kuantum dünyasının "sisli" ve "belirsiz" doğasıyla mükemmel uyum sağlar. Menülerde,

arka planı bulanıklaştıran yarı saydam paneller, yumuşak gölgeler ve neon vurgular (Camgöbeği/Magenta) kullanılarak fütüristik ancak temiz bir görünüm elde edilmelidir.[34]

---

6. Proje Yönetimi ve Uygulama Yol Haritası

Bu raporun teknik analizleri ışığında, projenin hayata geçirilmesi için önerilen teknoloji yığını (Tech Stack) ve uygulama adımları şöyledir:

6.1 Teknoloji Yığını (Tech Stack) Seçimi

Bileşen	Öneri	Gerekçe
Oyun Motoru	**Godot 4.5 (.NET/C#)**	Açık kaynak, lisans ücreti yok, tamamen yenilenmiş Stencil Buffer desteği, küçük APK boyutu, hızlı iterasyon.[5]
Backend	**Nakama (Heroic Labs)**	Go ile yazılmış yüksek performanslı sunucu, "Ghost Data" depolaması için ideal Storage Engine, açık kaynak.[7]
Veritabanı	**CockroachDB**	Nakama ile tam uyumlu, ölçeklenebilir SQL veritabanı.
Scripting	**C# (Godot)**	Tip güvenliği (Type Safety), performans ve Unity'den gelen geliştiriciler için kolay geçiş.[36]
Version Control	**Git (LFS)**	Büyük asset ve binary dosyaların yönetimi için Large File Storage (LFS) kullanımı zorunludur.

6.2 Performans Bütçesi ve Optimizasyon Hedefleri

Mobil cihazlarda (özellikle orta segment Android) kararlı 60 FPS deneyimi için şu sınırlar

aşılmamalıdır:

- **Draw Calls:** Maksimum 150-200. "Geometry Instancing" ve "Batching" teknikleri agresif şekilde kullanılmalıdır.[37]
- **Polygon Count:** Ekranda aynı anda maksimum 50.000 - 100.000 üçgen. Low-poly stilizasyon tercih edilmelidir.
- **Shader Complexity:** Mobil GPU'larda (Mali, Adreno) ALU (Arithmetic Logic Unit) işlemlerini azaltmak için, karmaşık matematiksel hesaplamalar Vertex Shader'a taşınmalı, Fragment Shader mümkün olduğunca hafif tutulmalıdır.[39]
- **Memory Bandwidth:** Stencil Buffer ve Post-Processing efektleri bant genişliğini tüketir. Render çözünürlüğü (Render Scale) dinamik olarak ayarlanmalı, gerekirse 0.8x ölçekleme kullanılmalıdır.

6.3 Risk Analizi

1. **Cihaz Parçalanması (Android Fragmentation):** Binlerce farklı Android cihazda shaderların (özellikle AGSL ve Vulkan) tutarlı çalışması zordur. Geniş kapsamlı cihaz testleri ve "Compatibility" render modu fallback'leri hazırlanmalıdır.[40]
2. **Determinizm Sorunu:** Ghost verilerinin farklı cihazlarda kayması (desync). Bu risk, "Durum Kaydı" (State Recording) yöntemi ve sıkı pozisyon düzeltme (Snap to position) algoritmaları ile minimize edilmelidir.

7. Sonuç

"Schrödinger'in Kedisi" konsepti, mobil oyun pazarında hem anlatısal hem de mekaniksel bir yenilik sunma potansiyeline sahiptir. Godot 4.5'in sunduğu ileri grafik özellikleri (Stencil Buffer) ve Nakama'nın güçlü backend altyapısı, bu karmaşık "Çift Dünya" deneyiminin mobil cihazlarda yüksek performansla çalışmasını teknik olarak mümkün kılmaktadır. Diegetic arayüz tasarımı ve tek dokunuşlu kontrollerle desteklenen bu proje, 2026 yılının rekabetçi pazarında öne çıkabilecek, sürdürülebilir ve genişletilebilir bir oyun mimarisi vaat etmektedir.

Alıntılanan çalışmalar

1. 2026 State of Mobile - Sensor Tower, erişim tarihi Şubat 8, 2026, https://sensortower.com/blog/state-of-mobile-2026
2. Schrödinger's cat - Wikipedia, erişim tarihi Şubat 8, 2026, https://en.wikipedia.org/wiki/Schr%C3%B6dinger%27s_cat
3. This quirky cat game has sparked a huge scientific debate - Creative Bloq, erişim tarihi Şubat 8, 2026, https://www.creativebloq.com/3d/video-game-design/this-quirky-cat-game-has-sparked-a-huge-scientific-debate
4. Performance problems I found · Issue #7366 · godotengine/godot-proposals - GitHub, erişim tarihi Şubat 8, 2026, https://github.com/godotengine/godot-proposals/issues/7366
5. RDPipelineDepthStencilState — Godot Engine (4.5) documentation ..., erişim tarihi

Şubat 8, 2026, https://docs.godotengine.org/en/4.5/classes/class_rdpipelinedepthstencilstate.html
6. Understanding URP settings and essentials - Unity - YouTube, erişim tarihi Şubat 8, 2026, https://www.youtube.com/watch?v=HCXCmHgV7Sk
7. Nakama vs Firebase - Heroic Labs, erişim tarihi Şubat 8, 2026, https://heroiclabs.com/comparison/firebase/
8. Architecture Overview - Heroic Labs Documentation, erişim tarihi Şubat 8, 2026, https://heroiclabs.com/docs/nakama/getting-started/architecture/
9. What did Schrodinger's Cat experiment prove? | Science Questions with Surprising Answers, erişim tarihi Şubat 8, 2026, https://www.wtamu.edu/~cbaird/sq/2013/07/30/what-did-schrodingers-cat-experiment-prove/
10. Collapse and Consciousness: Resolving Schrödinger's Cat Through Reversibility and Entropy | by Bill Giannakopoulos | Medium, erişim tarihi Şubat 8, 2026, https://medium.com/@bill.giannakopoulos/collapse-and-consciousness-resolving-schr%C3%B6dingers-cat-through-reversibility-and-entropy-a1c3f71afc28
11. Unity Shaders | Stencil Buffer Portals : r/Unity3D - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/Unity3D/comments/eaqhmp/unity_shaders_stencil_buffer_portals/
12. Stencil support to spatial materials in Godot 4.5 - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/godot/comments/1lbwkrd/stencil_support_to_spatial_materials_in_godot_45/
13. URP Renderer Feature | Universal RP | 8.3.1 - Unity - Manual, erişim tarihi Şubat 8, 2026, https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@8.3/manual/urp-renderer-feature.html
14. Configure for better performance in URP - Unity - Manual, erişim tarihi Şubat 8, 2026, https://docs.unity3d.com/6000.3/Documentation/Manual/urp/configure-for-better-performance.html
15. Adjust settings to improve performance in URP - Unity - Manual, erişim tarihi Şubat 8, 2026, https://docs.unity3d.com/6000.3/Documentation/Manual/urp/optimize-for-better-performance.html
16. Create a Scary Glitch Shader with Shader Graph Part 1 - YouTube, erişim tarihi Şubat 8, 2026, https://www.youtube.com/watch?v=W3CmMZ3ZFhg
17. How To Use Every Node in Unity Shader Graph - Daniel Ilett, erişim tarihi Şubat 8, 2026, https://danielilett.com/2021-05-20-every-shader-graph-node/
18. GPU-Accelerated Effects: Glitch at Scale | by Konstantin Zolotov ..., erişim tarihi Şubat 8, 2026, https://medium.com/@konstantinzolotov/gpu-accelerated-effects-glitch-at-scale-e59216afd1e8

19. FREE Pixelated Screen Effect in Unity URP (Shader Graph Tutorial) - YouTube, erişim tarihi Şubat 8, 2026, https://www.youtube.com/watch?v=e7cCIUlGvok
20. 'Ghost' that copies player movement from previous attempt : r/gamedev - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/gamedev/comments/10d1pwg/ghost_that_copies_player_movement_from_previous/
21. Optimizing your game | Netcode for Entities | 1.5.1 - Unity - Manual, erişim tarihi Şubat 8, 2026, https://docs.unity3d.com/Packages/com.unity.netcode@1.5/manual/optimizations.html
22. Universal Ghost Replay System - Community Resources - Developer Forum | Roblox, erişim tarihi Şubat 8, 2026, https://devforum.roblox.com/t/universal-ghost-replay-system/4014417
23. Best Mobile Game Backend Providers in 2026 - Slashdot, erişim tarihi Şubat 8, 2026, https://slashdot.org/software/mobile-game-backend-providers/
24. An example of a nakama server using 2022's bleeding edge TypeScript tooling - GitHub, erişim tarihi Şubat 8, 2026, https://github.com/orta/nakama-typescript-example
25. Nakama Server Runtime Code Project Setup using TypeScript - YouTube, erişim tarihi Şubat 8, 2026, https://www.youtube.com/watch?v=FXguREV6Zf8
26. Modeling for Scalability - Heroic Labs Documentation, erişim tarihi Şubat 8, 2026, https://heroiclabs.com/docs/nakama/concepts/storage/modeling/
27. Best Examples in Mobile Game UI Designs (2026 Review) - Pixune Studios, erişim tarihi Şubat 8, 2026, https://pixune.com/blog/best-examples-mobile-game-ui-design/
28. Top 5 Best Video Game UIs. Celebrating the greatest user interface... | by Akhil Dakinedi | SUPERJUMP | Medium, erişim tarihi Şubat 8, 2026, https://medium.com/super-jump/top-5-best-video-game-uis-db941d6a9357
29. The Minimal HUD Paradox: How Dreams of Diegetic Game Interfaces Often Lead to Cluttered Nightmares | by Ahmed Salama | Medium, erişim tarihi Şubat 8, 2026, https://medium.com/@salamatizm/the-minimal-hud-paradox-how-dreams-of-diegetic-game-interfaces-often-lead-to-cluttered-nightmares-e9cf7fae9d73
30. Diegetic gameplay elements gone wrong : r/TwoBestFriendsPlay - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/TwoBestFriendsPlay/comments/1dluiaq/diegetic_gameplay_elements_gone_wrong/
31. Split Masters - App Store, erişim tarihi Şubat 8, 2026, https://apps.apple.com/us/app/split-masters/id1103879308
32. Made a puzzle game where the controllers are inside of the game : r/IndieDev - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/IndieDev/comments/1my0gam/made_a_puzzle_game_where_the_controllers_are/
33. Game Design UX Best Practices - UX Planet, erişim tarihi Şubat 8, 2026, https://uxplanet.org/game-design-ux-best-practices-the-ultimate-guide-4a3078c32099

34. Emerging UI/UX Mobile Design Trends Redefining User Experience, erişim tarihi Şubat 8, 2026, https://dev-story.com/blog/mobile-app-ui-ux-design-trends/
35. Godot vs Unity - Which to Choose in 2025? - GameFromScratch.com, erişim tarihi Şubat 8, 2026, https://gamefromscratch.com/godot-vs-unity-which-to-choose-in-2025/
36. What's pushing you to consider switching from Godot to Unity/UE? - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/godot/comments/1kv011n/whats_pushing_you_to_consider_switching_from/
37. Optimization using batching — Godot Engine (3.5) documentation in English, erişim tarihi Şubat 8, 2026, https://docs.godotengine.org/en/3.5/tutorials/performance/batching.html
38. for more experienced devs is this alot of primitives, draw calls and objects? : r/godot - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/godot/comments/1hv2gjd/for_more_experienced_devs_is_this_alot_of/
39. Unity 6 tips: URP settings for mobile #gamedev #unitytips #unity3d #unitylearn #3dgamedevelopment - YouTube, erişim tarihi Şubat 8, 2026, https://www.youtube.com/shorts/Wgor3ZM00pw
40. Godot 4.5: Performance Fix Tested! Does It Deliver? - Reddit, erişim tarihi Şubat 8, 2026, https://www.reddit.com/r/godot/comments/1lfi0en/godot_45_performance_fix_tested_does_it_deliver/
41. Realtime Mobile Performance Logging | by Michael Short | spaceapetech - Medium, erişim tarihi Şubat 8, 2026, https://medium.com/spaceapetech/realtime-mobile-performance-logging-1e319277d970